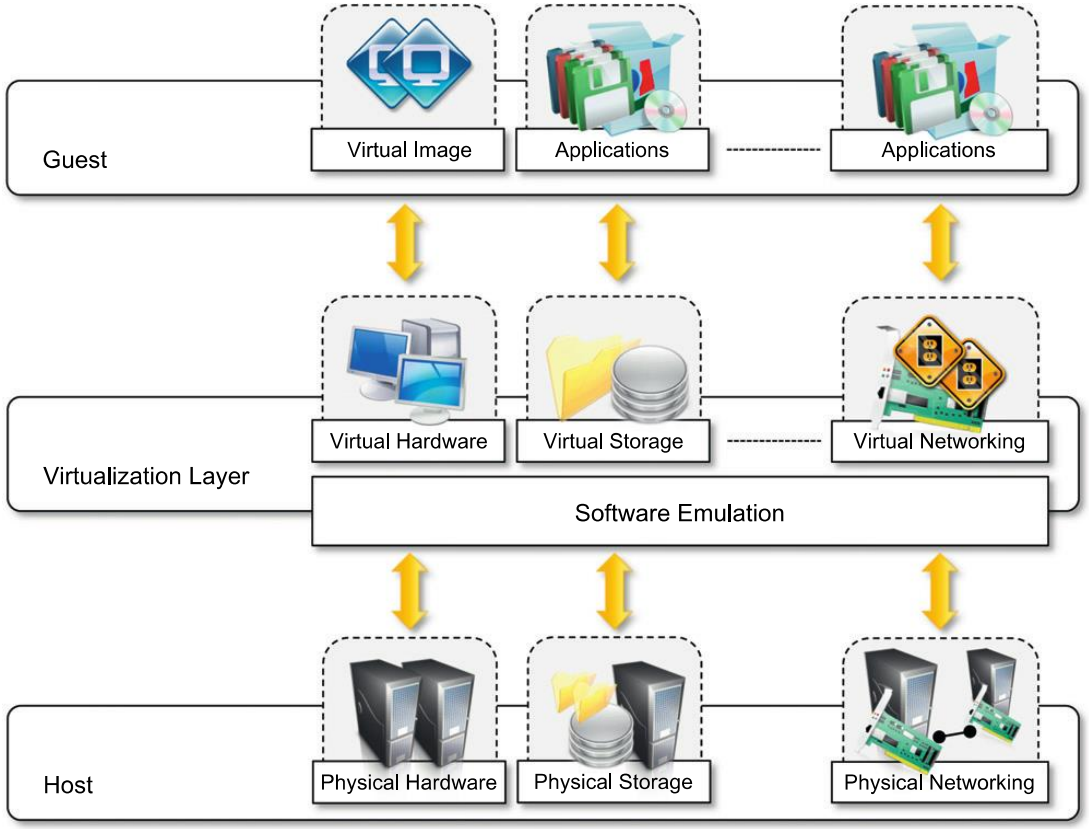
Cloud Computing

Lecture 2 - Enabling technologies - Virtualization

Emiliano Casalicchio
Department of Computer Science
Sapienza University of Rome

Characteristics of virtualization environments



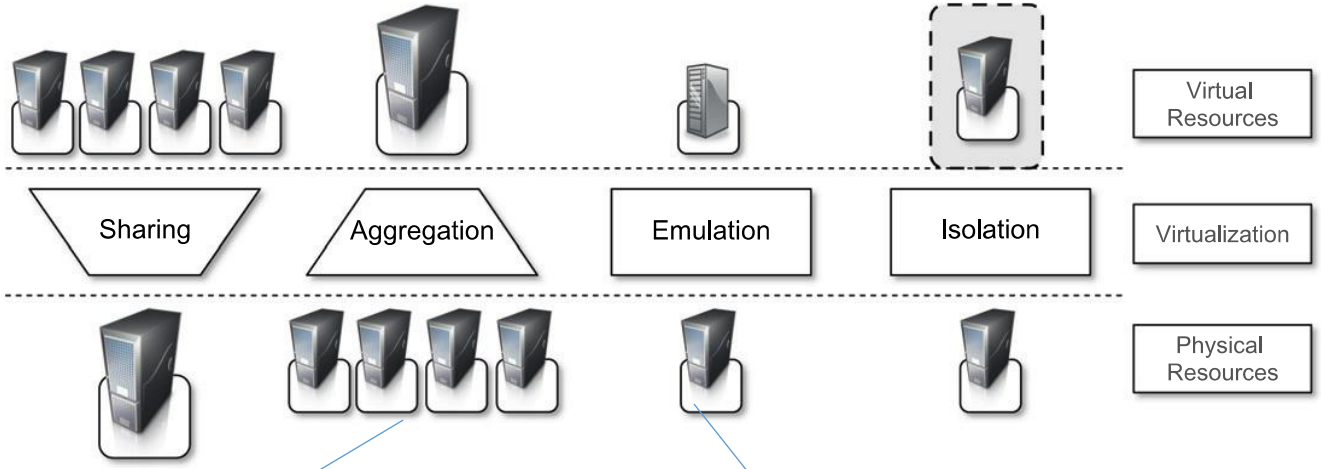
- Increased security
- Managed execution
- Portability

Virtual Machine Monitor (VMM) or Hypervisor

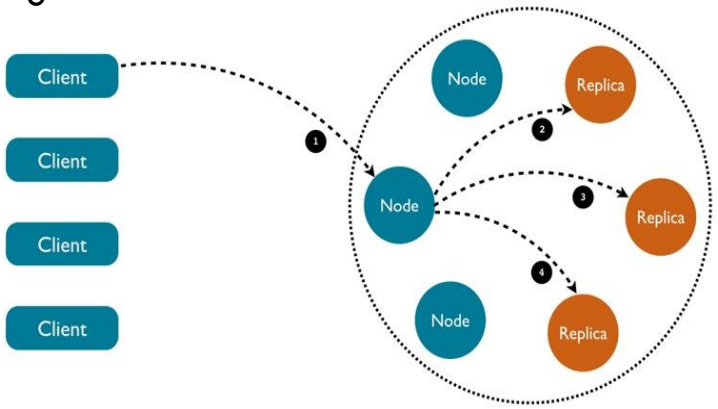
Increased security

- Why Virtualization increase security?
- The virtual machine manager control and filter the activity of the guest, thus preventing some harmful operations from being performed
  - Resources exposed by the host can then be hidden or simply protected from the guest
  - Sensitive information that is contained in the host can be naturally hidden without the need to install complex security policies
- Examples
  - Applets downloaded from the Internet run in a sandboxed version of the *Java Virtual Machine (JVM).* The sandbox filters harmful instructions on the bases of security policies
  - In Hardware virtualization the file system exposed by the VM is completely separated from the file system of the host

Managed execution



e.g. Cassandra distributed data store



Portability

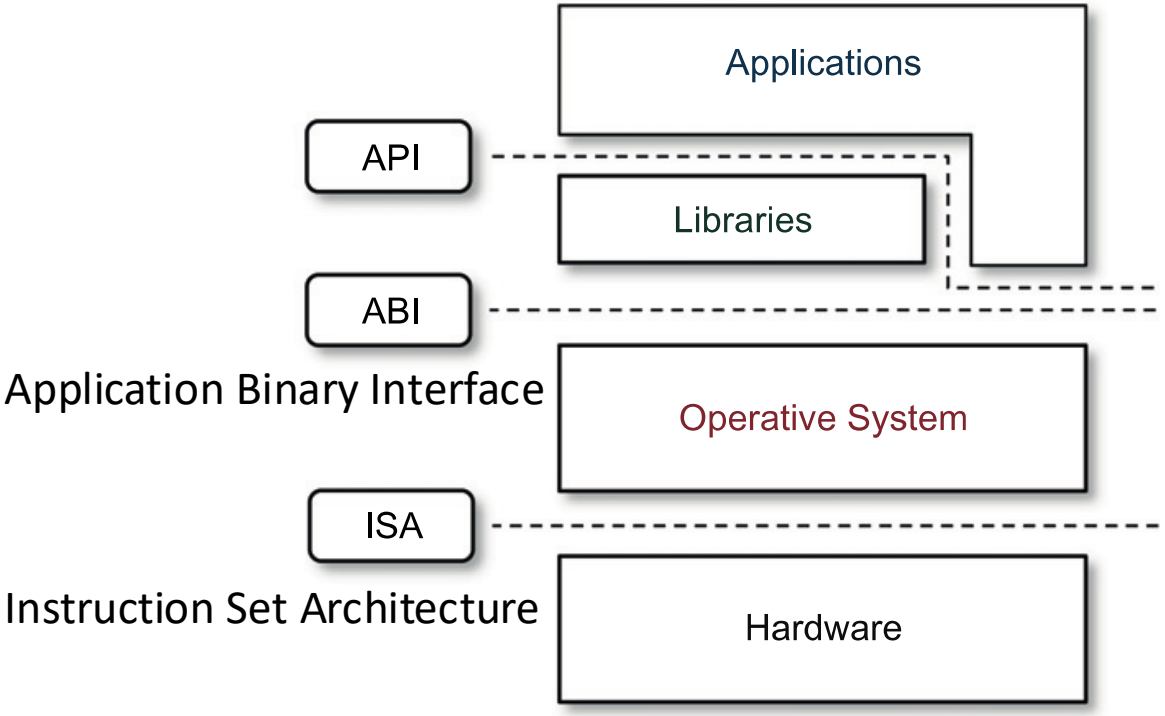
- Portability is the possibility to transfer and to use data and applications from a computing platform to another
- Why virtualization increase portability?

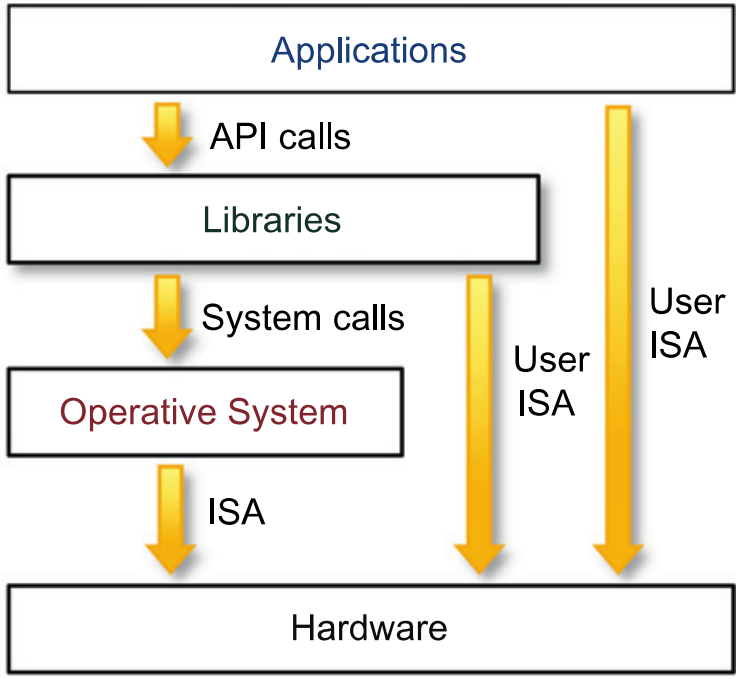
Portability

- A java or python code can be executed on any environment running a JVM or Python VM
- A Docker container can run on any platform running the Docker engine.
- A VM is booted from a VM image that is stored in a specific disk format
  - The disk format of a virtual machine image is the format of the underlying disk image, e.g.
    - aki, ari, ami, vdi, vhd, vhdx, vmdk, …
  - You can move around VM keeping or converting their format.

Machine reference model

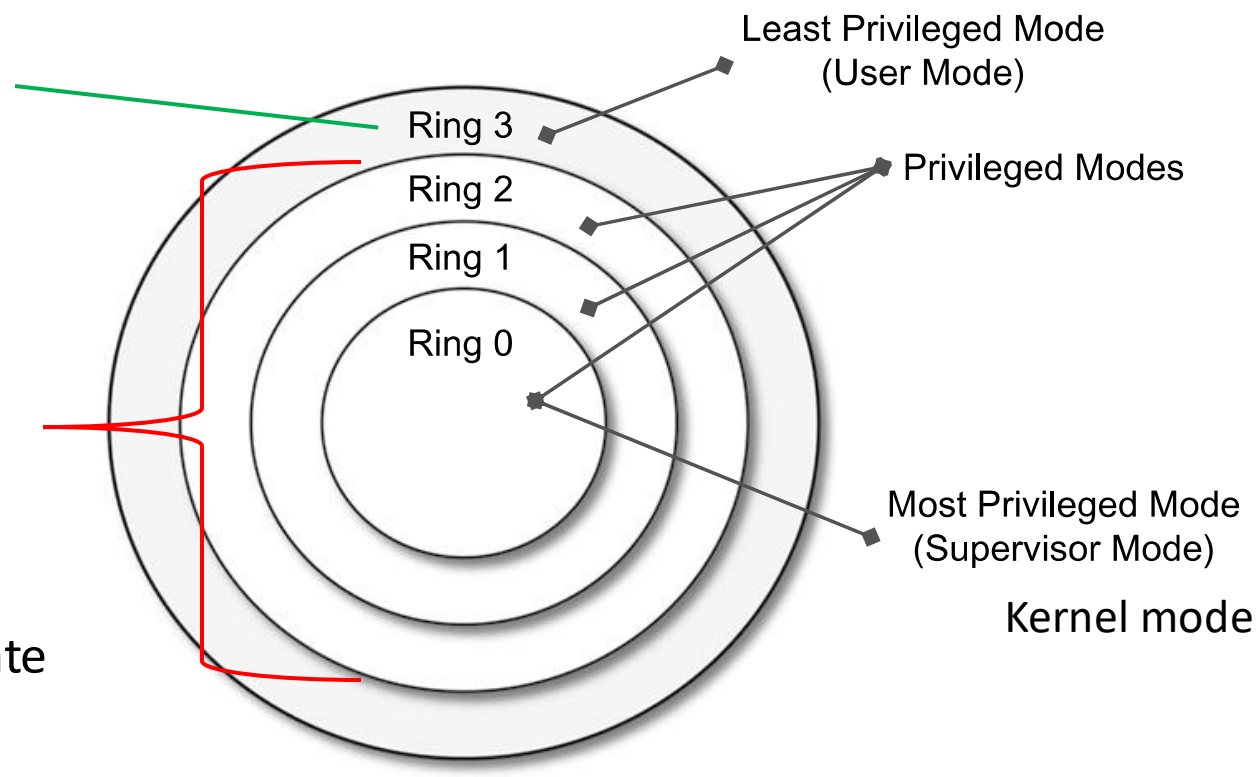
- To understand how the different virtualization techniques work it is fundamental to recall the layered structure of a computer system





Privileged and Nonprivileged instructions

- **Nonprivileged** instructions - can be used without interfering with other tasks
  - floating, fixed-point, and arithmetic instructions.
- **Privileged** instructions - are executed under specific restrictions and are mostly used for sensitive operations
  - behavior-sensitive instructions are those that operate on the I/O
  - control-sensitive instructions alter the state of the CPU registers
- Ring 1 and 2 are not used in modern systems



Hyper-visor

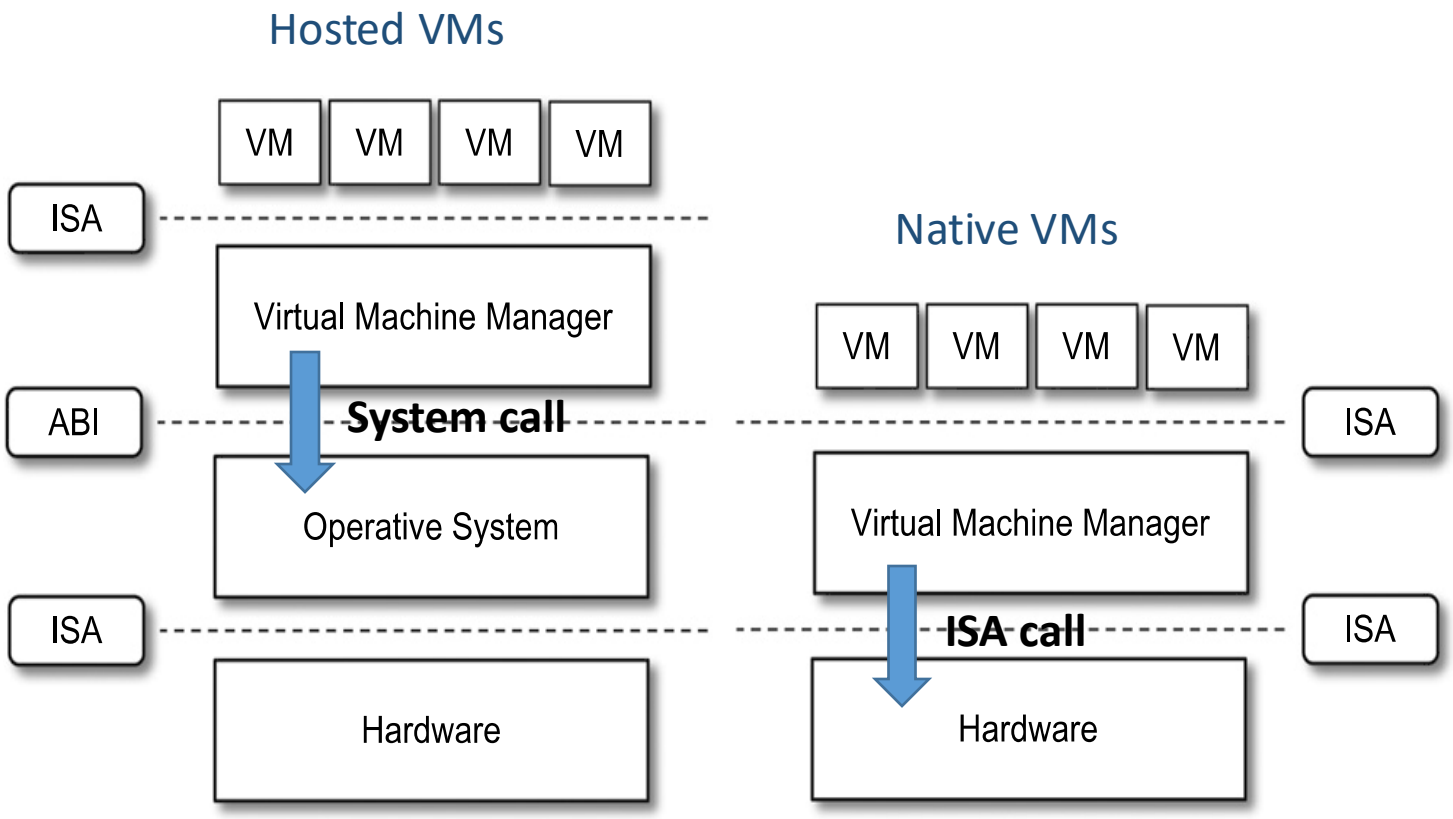
- The hyper-visor run on top of the supervisor mode, hence *hyper*
  - In practice, hypervisor run in supervisor mode

- Challenge: to fully emulate and manage the status of the CPU for guest operating systems all the sensitive instructions should be executed in privileged mode, which requires supervisor (kernel) mode

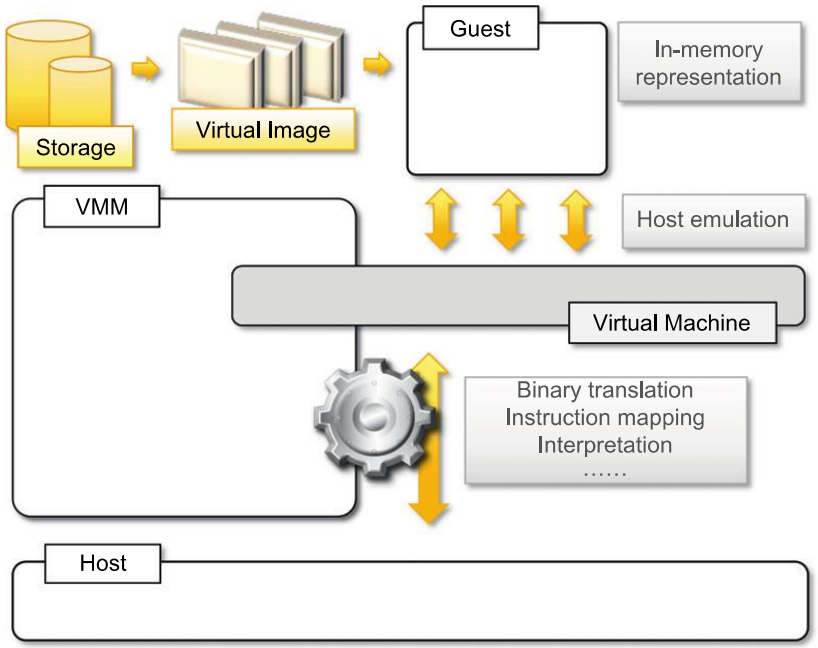
  - Not true for the original ISA (17 sensitive instructions called in user mode) → no isolation among guest OSs

  - With Intel VT and AMD V such instructions have been redesigned as privileged

Hardware (or system) level virtualization



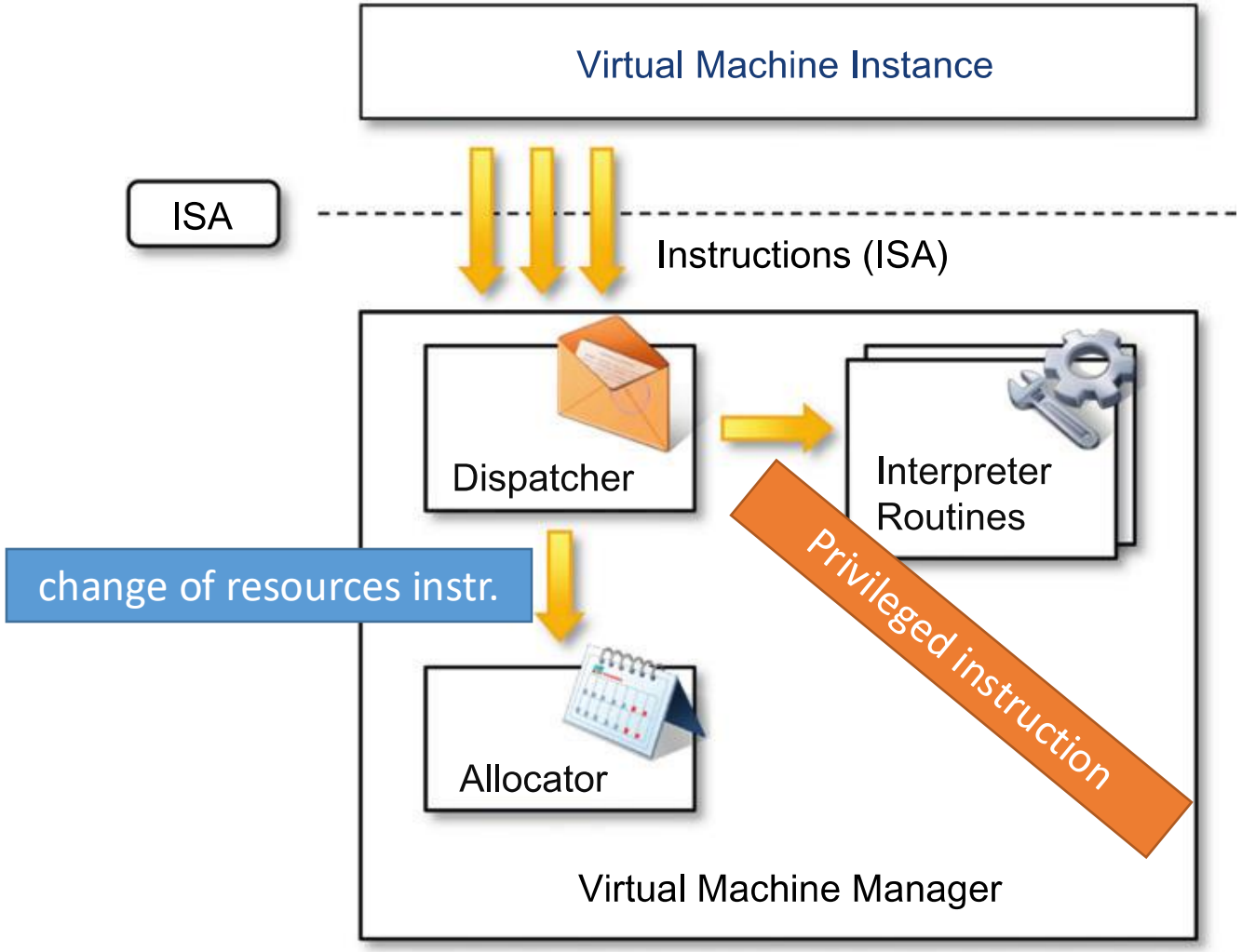
TYPE II Hypervisor **TYPE I Hypervisor**



Type 1 and Type 2 Hypervisors examples

Virtualization method	**Type 1 hypervisor**	**Type 2 hypervisor**
Virtualization without HW support	ESX Server 1.0	VMware Workstation 1
Paravirtualization	Xen 1.0	VirtualBox 5.0+
Virtualization with HW support	vSphere, Xen, Hyper-V	VMware Fusion, KVM, Parallels

Virtual Machine Manager



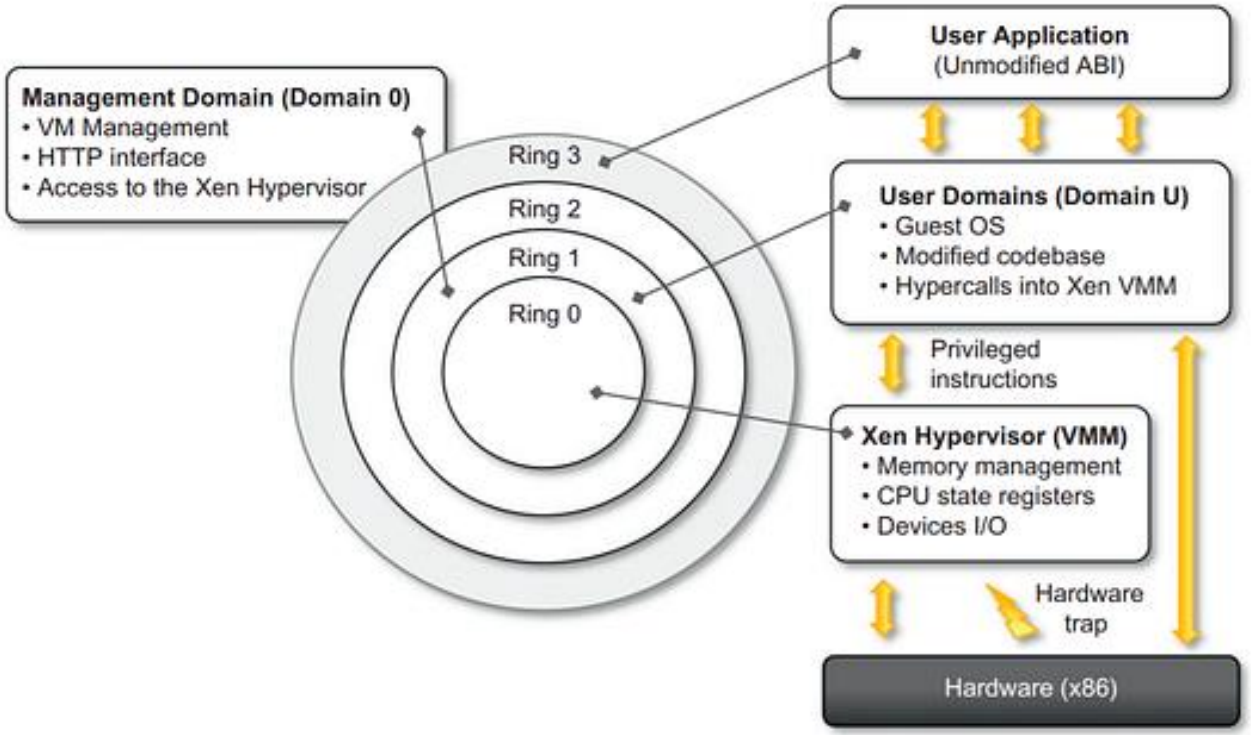
Three properties must be satisfied

- ***Equivalence.*** *A guest running under the control of a virtual machine manager should exhibit the same behavior as when it is executed directly on the physical host*
- ***Resource control.*** *The virtual machine manager should be in complete control of virtualized resources*
- ***Efficiency.*** *A statistically dominant fraction of the machine instructions should be executed without intervention from the virtual machine manager*

Popek GJ, Goldberg RP. Formal requirements for virtualizable third generation architectures. Commun ACM 1974;17(7)

Type I Hypervisor (and the Xen Architecture example)

- The VMM sit on bare metal
- The VMM can be a microkernel or a monolithic architecture
  - Microkernel: MS Hyper V, Xen
  - Monolithic: VMware ESX
- Microkernel VMM includes memory and CPU management – device drivers are in the host OS
- Monolithic VMM includes all the above
- Xen VMM implements Memory and CPU management, Devices I/O access
  - Provide hypercalls for sensitive instructions
  - Require to modify the guest OS – **Paravirtualization**
- The VM management is performed by the OS in domain 0
  - Allocate and map resources for domain 0
  - Create, copy, save, read, modify, share, migrate VMs



Trap on Privileged Instructions: how it works

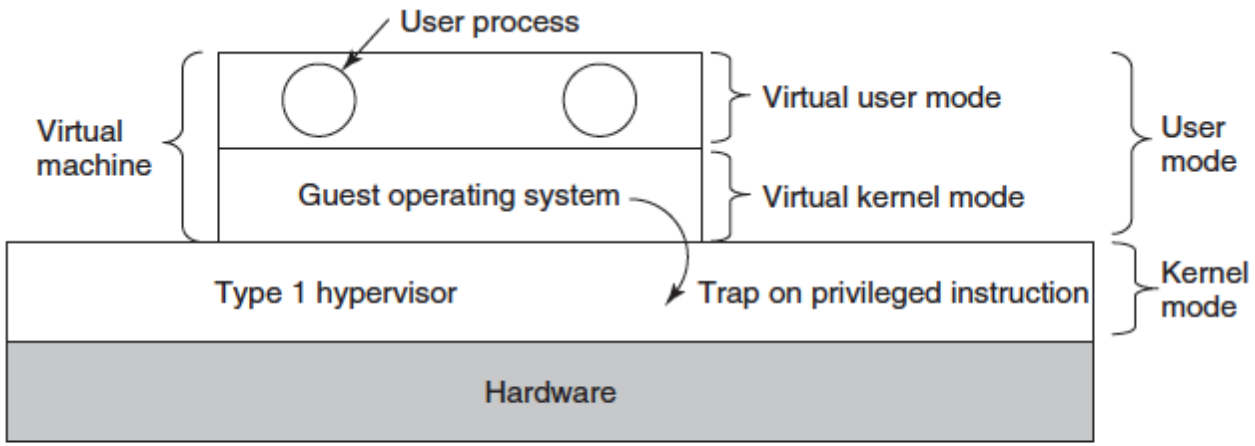
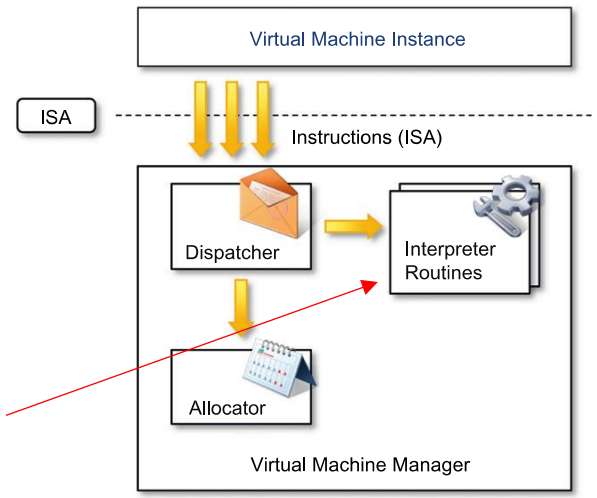


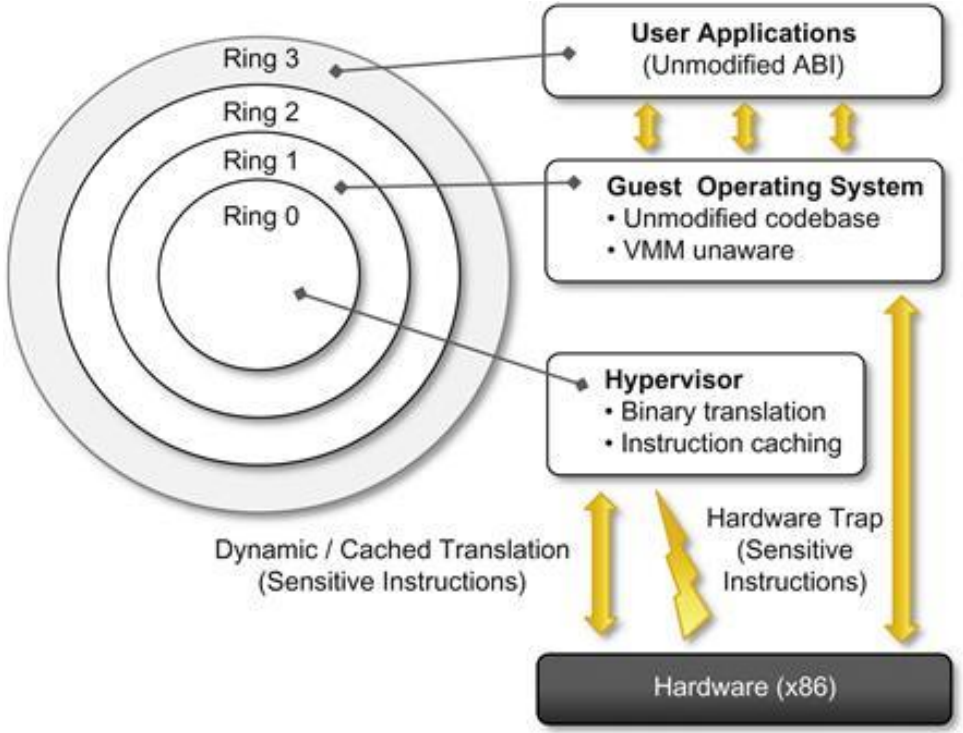
Figure 7-3. When the operating system in a virtual machine executes a kernel-only instruction, it traps to the hypervisor if virtualization technology is present.

- Guest OS executes an instruction allowed only when CPU is in kernel mode
  - X386 fails
  - VT a trap to the hypervisor does occur
- If a trap occur the Hypervisor inspect the instruction
  - Instruction from the guest OS → it arranges for the instruction to be carried out
  - Instruction from a user program → it emulates what the real hardware would do when confronted with a sensitive instruction executed in user mode

Full virtualization

- The VMM scans the instruction stream
  - Noncritical instructions run on the hardware directly
  - Privileged, control- and behavior-sensitive instructions (critical) are identified and trapped into the VMM, which emulates the behavior of these instructions
    - Binary translation, hypercall
- The guest OS is completely decoupled from the unde hardware
  - the guest OS is unaware that it is being virtualized



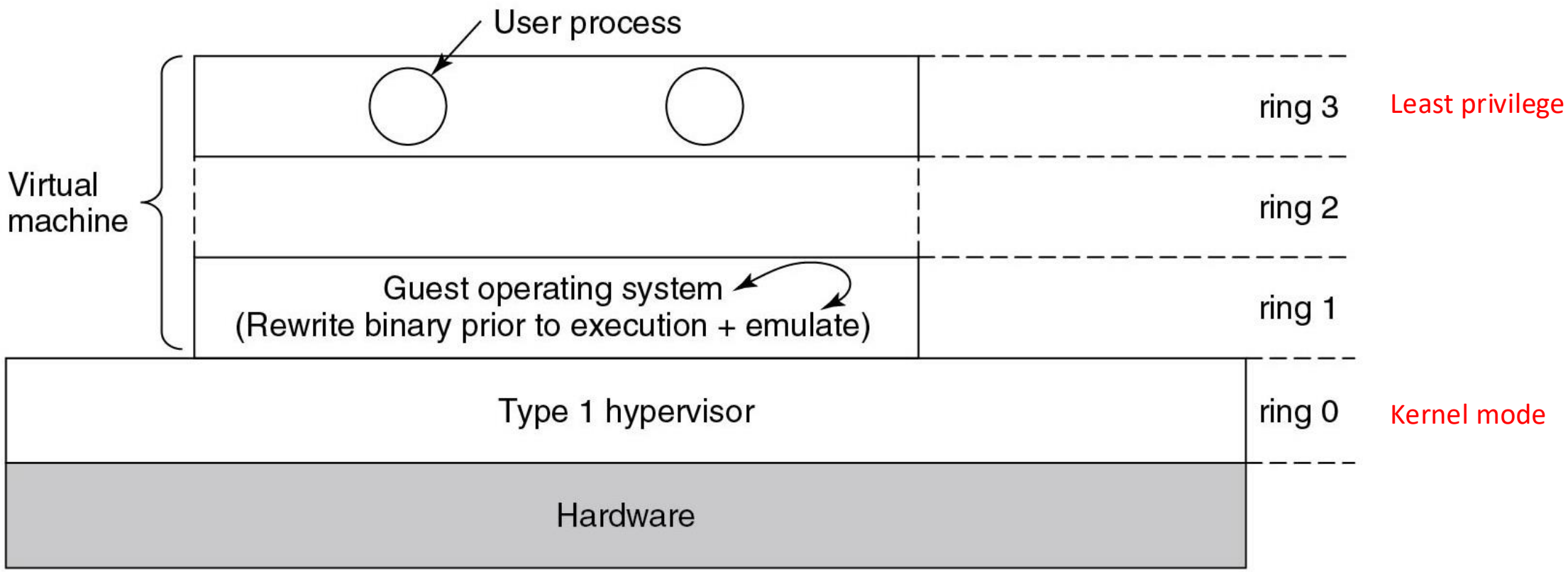


The Challenges of Virtualization on the x86

Major Challenges:

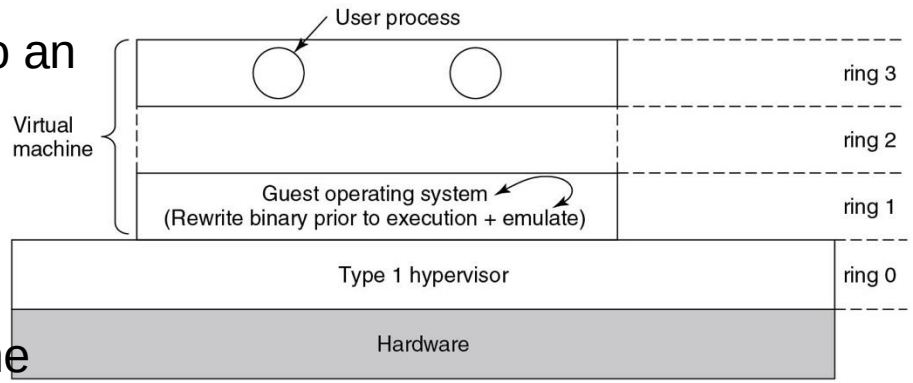
- The x86 architecture was not virtualizable
  - Sensible instructions are not a subset of privileged instructions
- The x86 architecture was of daunting complexity
- x86 machines had diverse peripherals
- Need for a simple user experience

Binary Translation



Binary Translation (2)

- Any attempt to access kernel memory from a user program leads to an access violation (trapped by guest OS)
- Guest operating system's **privileged instructions:**
  - trap to the hypervisor;
  - The hypervisor does some sanity checks and then performs the instructions on the guest's behalf
- Guest operating system's **sensitive instructions:**
  - The hypervisor rewrite the code, one basic block at a time
  - prior to executing a basic block the hypervisor
    - first scans it to see if it contains sensitive instructions
    - if so, replaces sensitive instructions with a call to a hypervisor procedure that handles them.
    - Replace the branch on the last instruction by a call into the hypervisor (to make sure it can repeat the procedure for the next basic block)



A **basic block** is a short, straight-line sequence of instructions that ends with a branch
- No jump, call, trap, return, or any flow alteration instructions

Pearson

Copyright © 2023, 2014, 2008 Pearson Education, Inc. All Rights Reserved

Binary Translation (3)

- Dynamic binary translation is not so expensive
  - Caching of translated blocks
  - Most code blocks do not contain sensitive or privilege instructions (can be executed natively, no translation)
  - Binary translator can ignore all user processes; they execute in user mode if sensitive instructions are executed the HW ignore them
  - All the code running in ring 1 (guest OS code) is all translated to avoid traps that are expensive to handle.

Pearson

Copyright © 2023, 2014, 2008 Pearson Education, Inc. All Rights Reserved

Hardware-supported Virtualization Technology

- In 2005, Intel introduced Virtualization Technology (VT-x)
- AMD followed with AMD-v
- Sensitive instructions are a subset of privileged instructions
- I/O instructions, etc, **trap to** the hypervisor which does the work on behalf of the virtual machine
  - A lot of trap slow down the performance vs binary translation

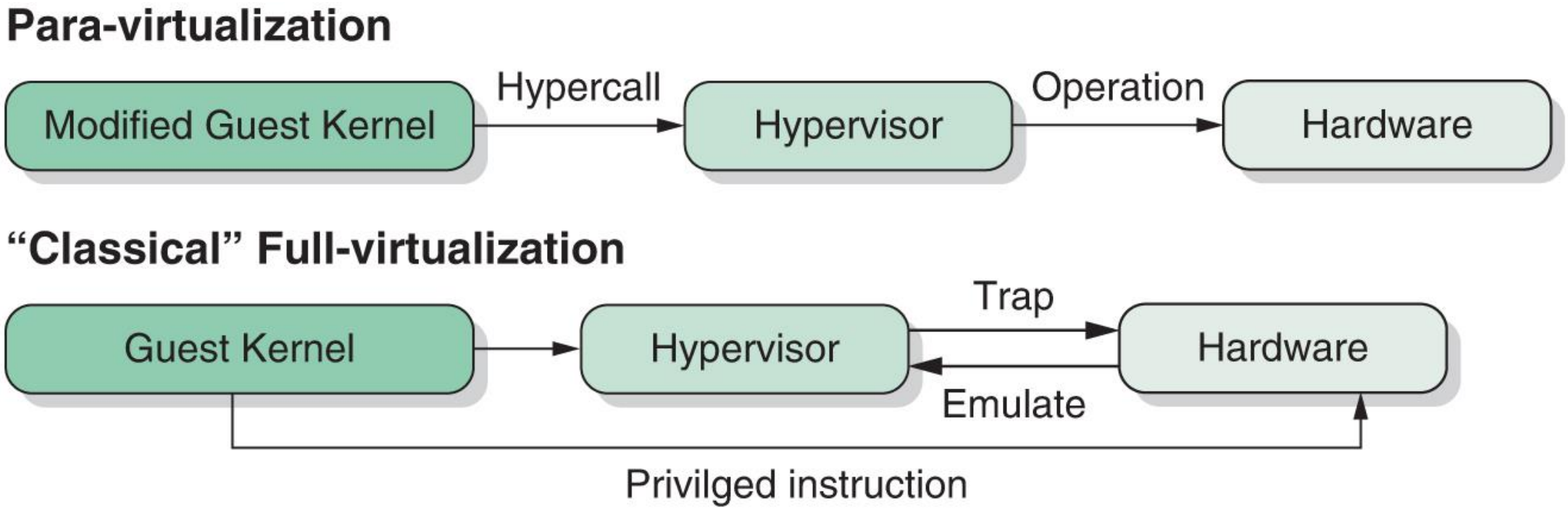
Pearson

Copyright © 2023, 2014, 2008 Pearson Education, Inc. All Rights Reserved

Paravirtualization

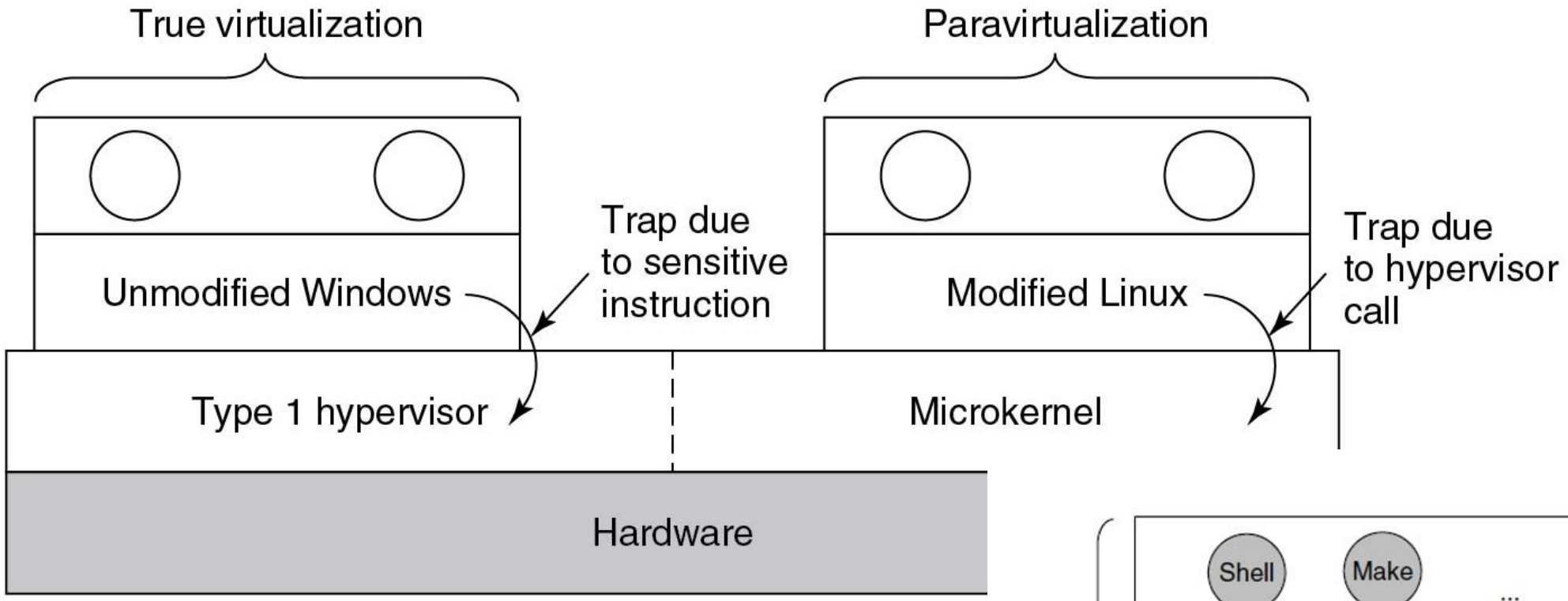
- Para-virtualized OS are assisted by an intelligent compiler to replace the nonvirtualizable OS instructions by hypercalls
  - The guest must support paravirtualization
  - Require HW assisted virtualization
  - Prevent performance losses
- Examples
  - KVM (linux)
  - Hyper-V

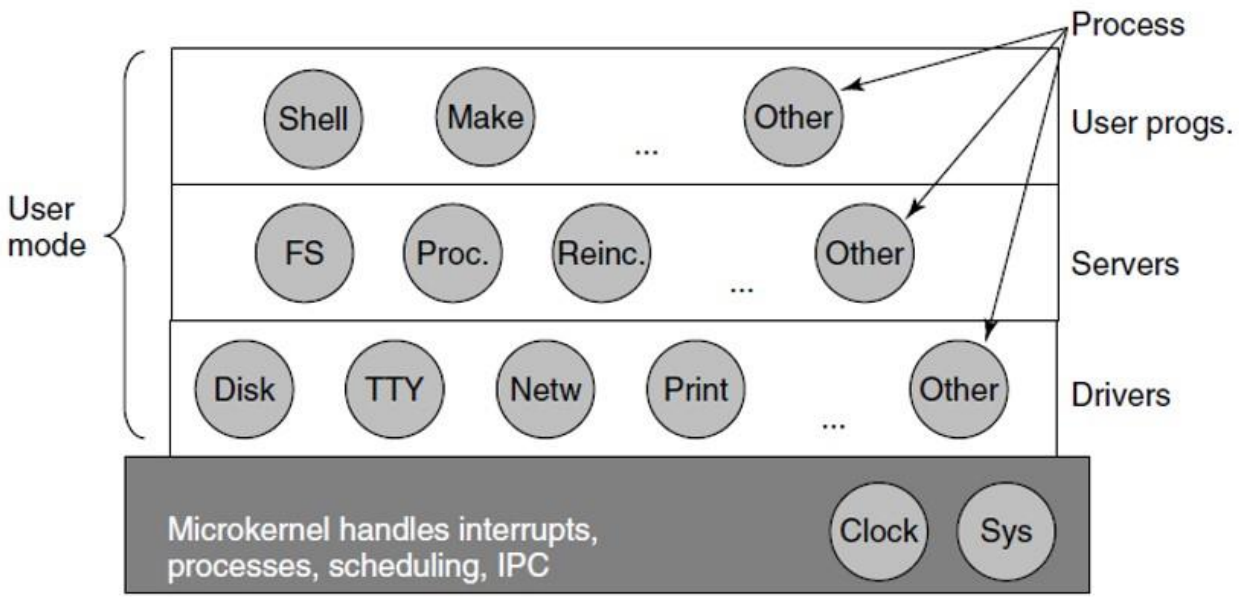
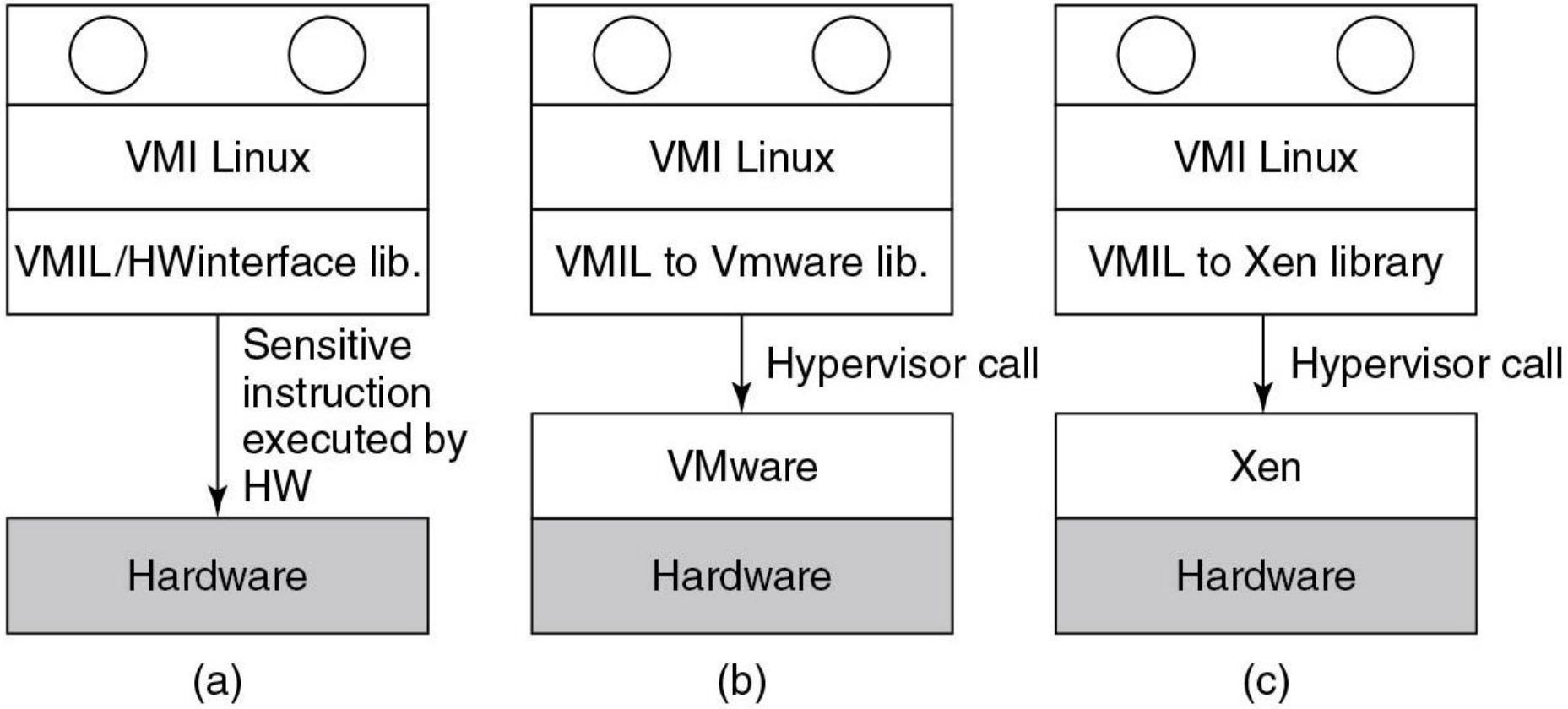
Paravirtualization



- Do not pretend to be hardware, make OS aware that it is in a VM
- Hypervisor expose an API that guest OS uses rather than syscalls
  - Sensitive instructions are all replaced by hypercalls
- This makes many things easier

Paravirtualization leverage Microkernels (1)





Paravirtualization leverage Microkernels (2)